



SWEN 221

A surreal, dreamlike landscape. In the center, a person stands on a small, dark, rocky outcrop that appears to be part of a vast, layered sky. The person is looking out over a horizon where a bright, golden light source, possibly a sun or moon, is setting or rising, casting a warm glow across the scene. The sky is filled with large, billowing clouds in shades of pink, purple, blue, and white. A bright comet streaks across the upper left portion of the sky. The overall atmosphere is ethereal and otherworldly.

Marco Servetto
CO258

www.youtube.com/MarcoServetto



Time Travel



Time Travel



**Good object oriented code can transcend time
and invert the usual flow of control**

Normal flow of time, normal flow of control

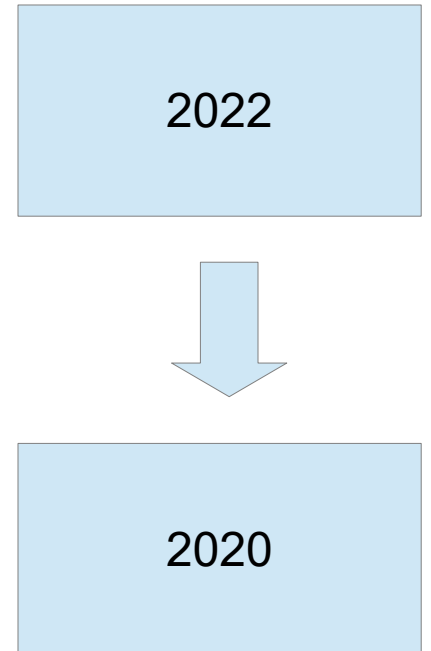
```
record Cat(String name,int age){//Cat from 2020
    @Override public String toString(){
        return ">^.^<"+name;
    }
}
```

2020

Normal flow of time, normal flow of control

```
record Cat(String name,int age){//Cat from 2020
    @Override public String toString(){
        return ">^.^<"+name;
    }
}

record Person(String name, Cat cat){//Person from 2022
    @Override public String toString(){
        return name + " & " + cat.toString();//Calls the code of Cat from 2020
    }
}
```



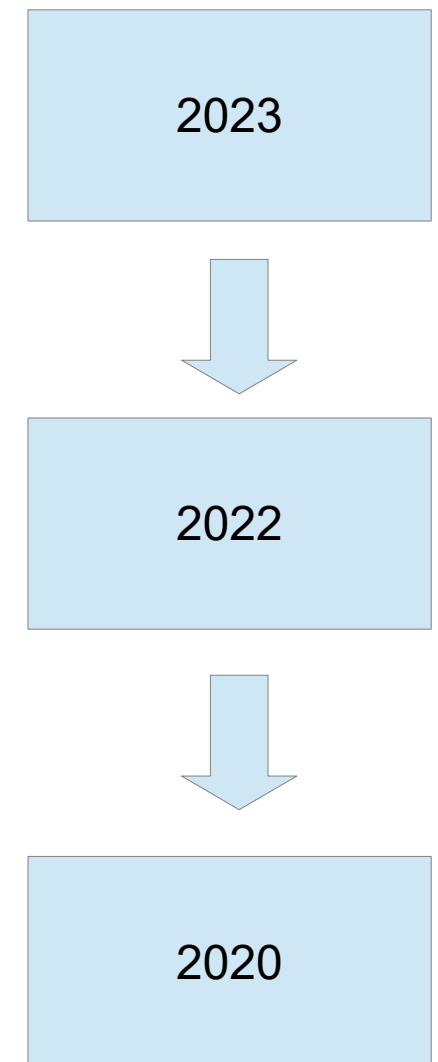
Code from the future can call code from the past.
Normal expected flow of control

Normal flow of time, normal flow of control

```
record Cat(String name,int age){//Cat from 2020
    @Override public String toString(){
        return ">^.^<"+name;
    }
}

record Person(String name, Cat cat){//Person from 2022
    @Override public String toString(){
        return name + " & " + cat.toString();//Calls the code of Cat from 2020
    }
}

class Main{//Main from 2023
    public static void main(String[] arg){
        var bob = new Person("Bob",new Cat("Meow",2)).toString();
        //Calls the code of Person from 2022
        //Person calls code from 2020
        System.out.println(bob);//Bob & >^.^<Meow
    }
}
```



Code from the future can call code from the past.
Normal expected flow of control

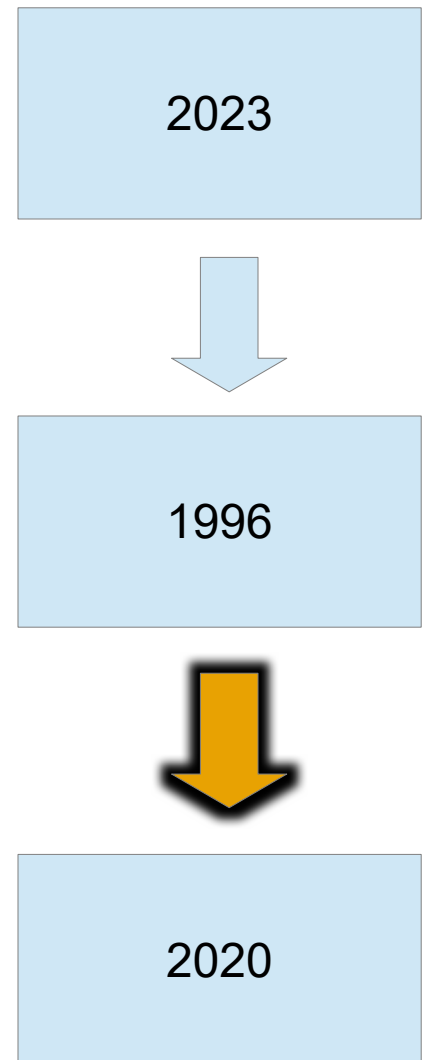
Time travel! Inverted flow of control

```
import java.util.List; //List was written in the far past (probably 1996)
```

```
record Cat(String name,int age){//Cat from 2020
    @Override public String toString(){
        return ">^.^<"+name;
    }
}
```

```
class Main{//written today (2023)
    public static void main(String[] arg){
        var cats= List.of(new Cat("Molly",4),new Cat("Meow",2)).toString();
        //Calls the code of List from 1996
        //List calls code from 2020!!
        System.out.println(cats);//[>^.^<Molly, >^.^<Meow]
    }
}
```

Code from the past calls code from (its) future.
Time travel!

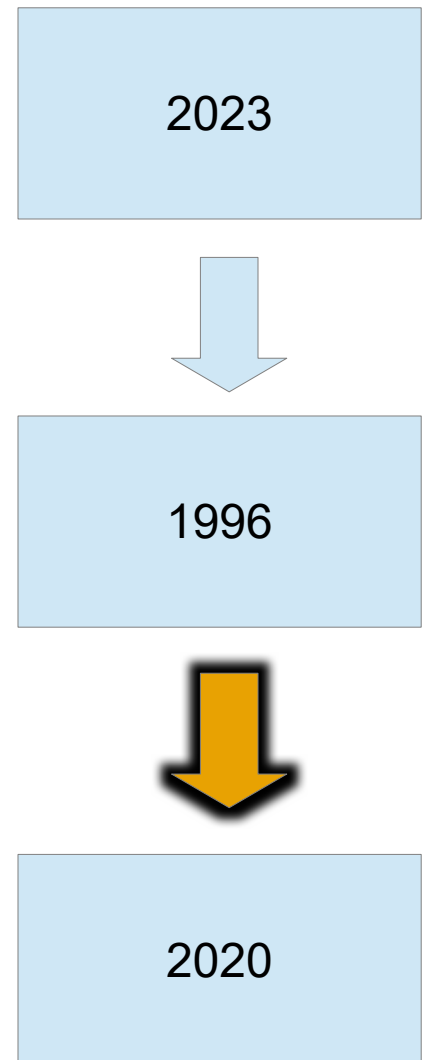


How can this be?

- The trick: list just knows about 'Object',
- Object was written before List.
- Cat extends Object.
- In this case, Object.toString works as a time wormhole

I call it "a time wormhole" to emphasize the idea that the calling code is able to access code written in the future through some kind of shortcut.

To convey the idea that the flow of control is able to bend or circumvent the normal time dependency flow.





We have seen this already!

```
interface HouseDrawing extends Drawing{
    default void draw(Graphics2D g){
        this.drawWall(g);
        this.drawWindows(g);
        this.drawDoor(g);
        this.drawRoof(g);
    }
    default void drawWall(Graphics2D g){
        g.setColor(Color.GRAY);
        g.fillRect(50,200,300,100);
    }
    default void drawWindows(Graphics2D g){
        g.setColor(Color.CYAN);
        g.fillRect(60,210,30,20);
        g.fillRect(60,240,30,20);
    }
    default void drawDoor(Graphics2D g){
        g.setColor(Color.RED);
        g.fillRect(150,240,30,60);
    }
    default void drawRoof(Graphics2D g){
        g.setColor(Color.BLACK);
        g.fillPolygon(new int[]{30,200,370},
                     new int[]{200,100,200},3);
    }
}
```

HouseDrawing was written
some time ago

But MarcoHouse written today
can override method drawWall

```
class MarcoHouse implements HouseDrawing{
    @Override
    public void drawWall(Graphics2D g){
        g.setColor(Color.BLUE);
        g.fillRect(50,200,400,100);
    }
}
```

Now the code of
HouseDrawing.draw can call
code written in the future!

We have seen this already!

```
interface HouseDrawing extends Drawing{
    default void draw(Graphics2D g){
        this.drawWall(g);
        this.drawWindows(g);
        this.drawDoor(g);
        this.drawRoof(g);
    }
    default void drawWall(Graphics2D g){
        g.setColor(Color.GRAY);
        g.fillRect(50,200,300,100);
    }
    default void drawWindows(Graphics2D g){
        g.setColor(Color.CYAN);
        g.fillRect(60,210,30,20);
        g.fillRect(60,240,30,20);
    }
    default void drawDoor(Graphics2D g){
        g.setColor(Color.RED);
        g.fillRect(150,240,30,60);
    }
    default void drawRoof(Graphics2D g){
        g.setColor(Color.BLACK);
        g.fillPolygon(new int[]{30,200,370},
                     new int[]{200,100,200},3);
    }
}
```

HouseDrawing was written
some time ago

But MarcoHouse written today
can override method drawWall

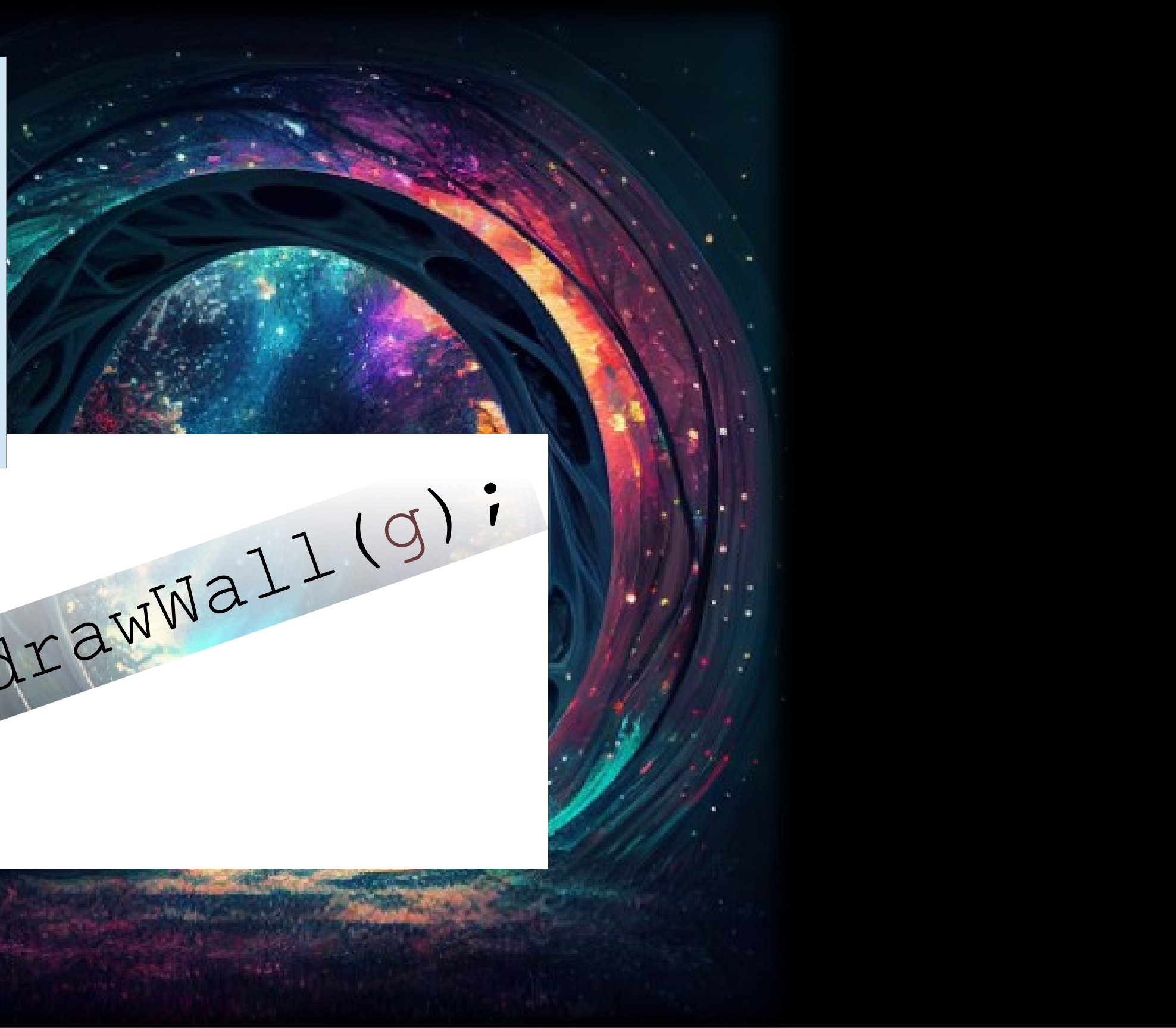
```
class MarcoHouse implements HouseDrawing{
    @Override
    public void drawWall(Graphics2D g){
        g.setColor(Color.BLUE);
        g.fillRect(50,200,400,100);
    }
}
```

Now the code of
HouseDrawing.draw can call
code written in the future!

The receiver
(**this**, in this case)
is the key
to time travel.



```
this.drawWall(g);
```



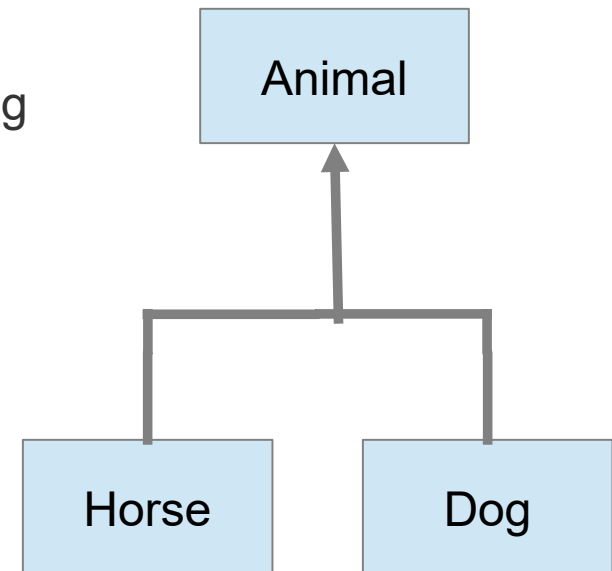
Static type and Dynamic type

- Every time the static type of a value is different from its dynamic type, time traveling can happen!
- **Static type:** the type of a value as seen by the compiler
 ‘this’ static type is HouseDrawing
- **Dynamic type:** the type of a value at run time
 ‘this’ dynamic type is MarcoHouse
- **Time traveling:** code from the past calls code from the future

Example hierarchy

```
interface Animal {  
    default void run(..){ . . . }  
}  
class Horse implements Animal{  
    . . .  
}  
class Dog implements Animal{  
    . . .  
}
```

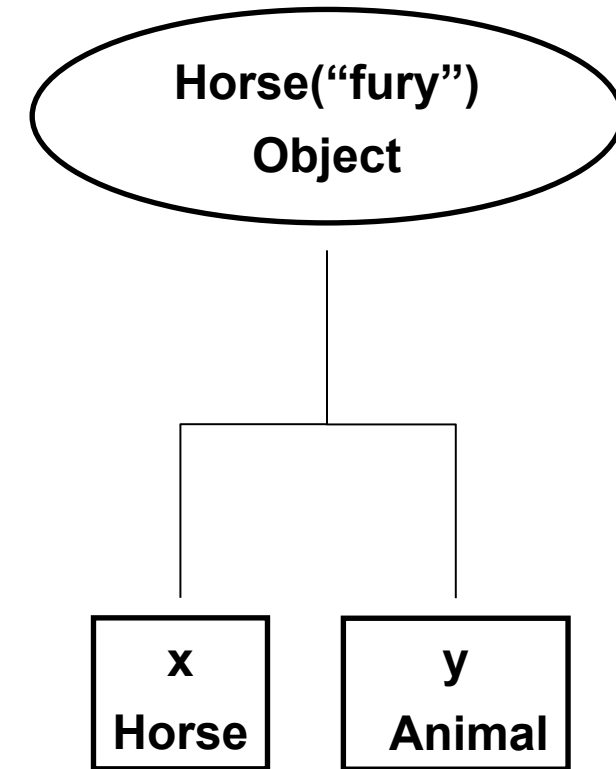
Superclass : Animal
Subclass: Horse, Dog



Mental Model of Typing

```
interface Animal {  
    default void run(..){ . . . }  
}  
class Horse implements Animal{  
    . . .  
}  
class Dog implements Animal{  
    . . .  
}
```

```
Horse x = new Horse("fury");  
Animal y = x;  
//y is still a horse.  
//But it is 'dressed' like an animal
```



Static (or Declared)
type of y is "Animal"

Dynamic (or Runtime)
type of x and y is
"Horse"

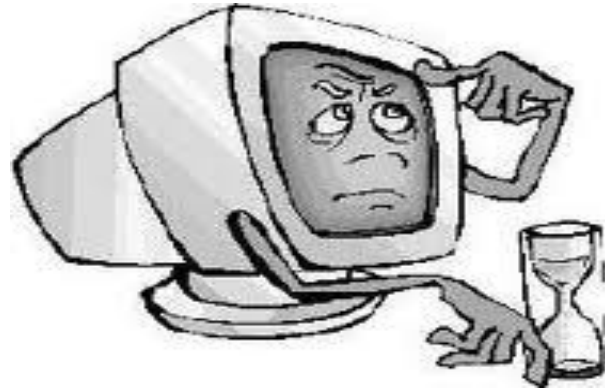
- Static Type:
 - Declared type of a variable
- Dynamic Type:
 - Type of object *referred* to by variable
 - potentially different in different moments

Again!

- Every time the static type of a value is different from its dynamic type, time traveling can happen!
- What is the static type?
- What is the dynamic type?
- What is time traveling?

```
Animal x = .....
```

```
x.run(myDest);
```



```
class Horse implements Animal{
```

```
..
```

```
  public double speed(){
```

```
    return 10d;
```

```
  }
```

```
..
```

```
}
```

```
interface Animal{
```

```
..
```

```
  default void run(Point2D dest){
```

```
    double s = this.speed();
```

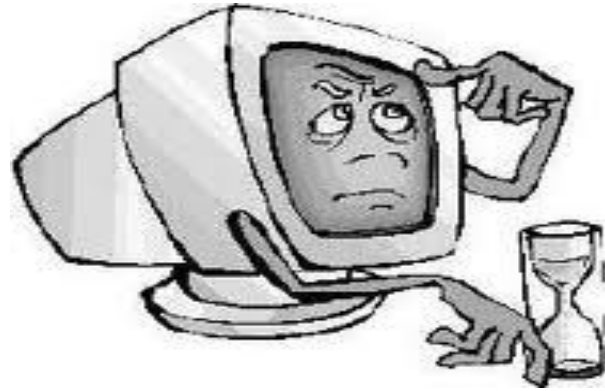
```
    ..
```

```
  }
```

```
}
```

```
Animal x = ....
```

```
x.run(myDest);
```



```
Animal x = new Horse(...);
```

```
x.run(myDest);
```



```
class Horse implements Animal{
```

```
..
```

```
public double speed(){
```

```
    return 10d;
```

```
}
```

```
..
```

```
}
```

```
interface Animal{
```

```
..
```

```
default void run(Point2D dest){
```

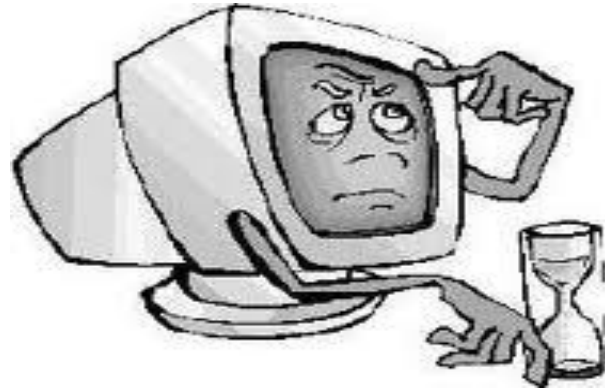
```
    double s = this.speed();
```

```
..
```

```
}
```

Is there time traveling
in this moment?

```
Animal x = ....  
x.run(myDest);
```



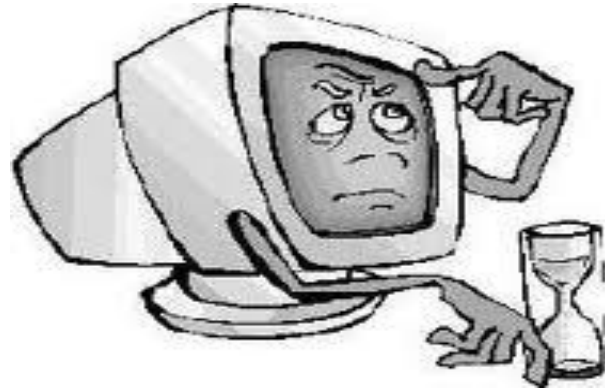
```
Animal x = new Horse(...);  
x.run(myDest);
```



```
class Horse implements Animal{  
    ..  
    public double speed(){  
        return 10d;  
    }  
    ..  
}  
  
interface Animal{  
    ..  
    default void run(Point2D dest){  
        double s = this.speed();  
        ..  
    }  
}
```

The selected implementation
is the one inside Animal
-Animal is the static type
No time travel

```
Animal x = ....  
x.run(myDest);
```



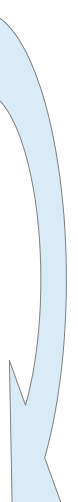
```
Animal x = new Horse(...);  
x.run(myDest);
```



```
Animal x = new Horse(...);  
  
x.run(myDest); <-- Animal.run  
... double s = this.speed();
```



```
class Horse implements Animal{  
    ..  
    public double speed(){  
        return 10d;  
    }  
    ..  
}  
  
interface Animal{  
    ..  
    default void run(Point2D dest){  
        double s = this.speed();  
        ..  
    }  
}
```



```
Animal x = ....
```

```
x.run(myDest);
```

```
... double s = this.speed();
```

```
class Horse implements Animal{
```

```
..
```

```
public double speed(){
```

```
    return 10d;
```

```
}
```

```
..
```

```
}
```

```
interface Animal{
```

```
..
```

```
default void run(Point2D dest){
```

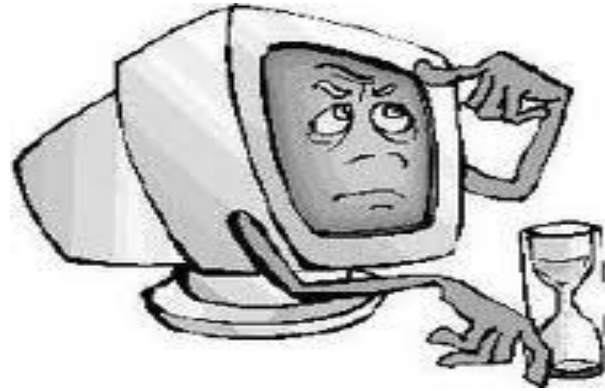
```
    double s = this.speed();
```

```
    ..
```

```
}
```

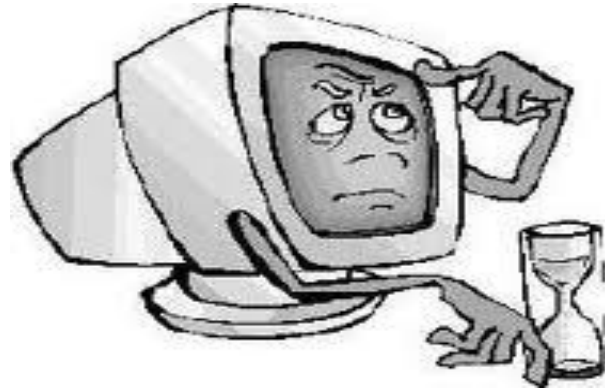
```
}
```

```
Animal x = ....  
x.run(myDest);  
... double s = this.speed();
```



```
class Horse implements Animal{  
    ..  
    public double speed(){  
        return 10d;  
    }  
    ..  
}  
  
interface Animal{  
    ..  
    default void run(Point2D dest){  
        double s = this.speed();  
        ..  
    }  
}
```

```
Animal x = ....  
x.run(myDest);  
... double s = this.speed();
```

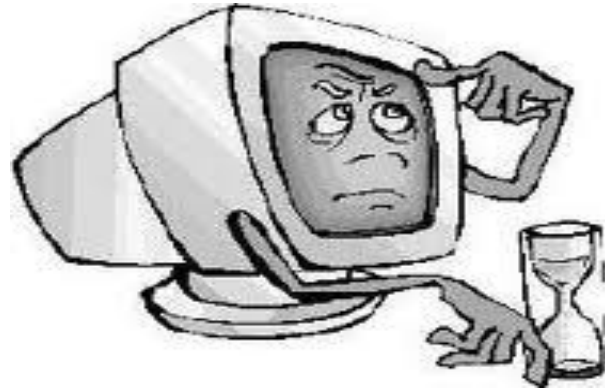


```
Animal x = new Horse(...);  
x.run(myDest);  
... double s = this.speed();
```



```
class Horse implements Animal{  
    ..  
    public double speed(){  
        return 10d;  
    }  
    ..  
}  
  
interface Animal{  
    ..  
    default void run(Point2D dest){  
        double s = this.speed();  
        ..  
    }  
}
```

```
Animal x = ....  
x.run(myDest);  
... double s = this.speed();
```



```
Animal x = new Horse(...);  
x.run(myDest);  
... double s = this.speed();
```



```
Animal x = new Horse(...);  
x.run(myDest);  
... double s = 10;
```

```
class Horse implements Animal{  
    ..  
    public double speed(){ ←  
        return 10d;  
    }  
    ..  
}  
  
interface Animal{  
    ..  
    default void run(Point2D dest){  
        double s = this.speed();  
        ..  
    }  
}
```

The selected implementation
is the one inside Horse
Time traveling may happen here!
Animal.run is calling Horse.speed

Time travelling pros/cons

- Pros: Open code
 - Future users of your code (this includes you) can make better use/reuse of your code.
 - Most OO programming patterns are based on dynamic dispatch.
 - (we will see later) Enables for better testing and Mock objects

Time travelling pros/cons

- Cons: open code

```
interface Point{ int x(); int y(); }  
...  
void usePoint(Point p){  
    System.out.println(p.x());  
}
```

- What can this code do?

Time travelling pros/cons

- Cons: open code

```
interface Point{ int x(); int y(); }  
...  
void usePoint(Point p){  
    System.out.println(p.x());  
}
```

- What can this code do?
- It could format the hard drive, for what I know!
- The future can come to hunt us down
 - The future is very, very big; and could get antagonistic and violent.
- Design for dynamic dispatch or prevent it!



Blocking time travelling: Final classes, final methods

- In Java the 'final' keyword means different things depending on where it is used.
- A final field or local variable can not be updated
- A final class can not be extended
 - prevents further time traveling from that type!
- A final method can not be overridden
 - prevents further time traveling from that method
- Record are always final!
- New: 'sealed' interfaces / classes are now available (see more later)

Exercises

```
record Person(String name, Cat cat){}
```

```
...  
void usePerson(Person bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - ????
- Q2 What is the dynamic type of bob?
 - ????
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - ????
 - ????
- Q4 Is there time travel in the call bob.name()?
 - ????

Exercises

```
record Person(String name, Cat cat){}
```

```
...  
void usePerson(Person bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - Person
- Q2 What is the dynamic type of bob?
 - ????
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - ????
 - ????
- Q4 Is there time travel in the call bob.name()?
 - ????

Exercises

```
record Person(String name, Cat cat){}
```

```
...  
void usePerson(Person bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - Person
- Q2 What is the dynamic type of bob?
 - Person (records are final)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - ????
 - ????
- Q4 Is there time travel in the call bob.name()?
 - ????

Exercises

```
record Person(String name, Cat cat){}
```

```
...  
void usePerson(Person bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - Person
- Q2 What is the dynamic type of bob?
 - Person (records are final)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - No, the method name() originates in Person (the static type)
- Q4 Is there time travel in the call bob.name()?
 - ????

Exercises

```
record Person(String name, Cat cat){}
```

```
...  
void usePerson(Person bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - Person
- Q2 What is the dynamic type of bob?
 - Person (records are final)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - No, the method name() originates in Person (the static type)
- Q4 Is there time travel in the call bob.name()?
 - No, name() is final / Person is final

No time travel is possible:
Person needs to exist
to compile usePerson.
usePerson can predict the
behavior of .name



Exercises

```
interface Person{  
    String name();  
    Cat cat();  
}
```

```
...  
void usePerson(Person bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - ????
- Q2 What is the dynamic type of bob?
 - ????
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - ????
 - ????
- Q4 Is there time travel in the call bob.name()?
 - ????
 - ????

Exercises

```
interface Person{  
    String name();  
    Cat cat();  
}
```

```
...  
void usePerson(Person bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - Person
- Q2 What is the dynamic type of bob?
 - ????
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - ????
 - ????
- Q4 Is there time travel in the call bob.name()?
 - ????
 - ????

Exercises

```
interface Person{  
    String name();  
    Cat cat();  
}  
  
...  
void usePerson(Person bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - Person
- Q2 What is the dynamic type of bob?
 - Who knows! (interface)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - ????
 - ???
- Q4 Is there time travel in the call bob.name()?
 - ????
 - ???

Exercises

```
interface Person{  
    String name();  
    Cat cat();  
}  
  
...  
void usePerson(Person bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - Person
- Q2 What is the dynamic type of bob?
 - Who knows! (interface)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - Yes, since name is abstract in Person (the static type)
- Q4 Is there time travel in the call bob.name()?
 - ????
 - ????

Exercises

```
interface Person{  
    String name();  
    Cat cat();  
}  
  
...  
void usePerson(Person bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - Person
- Q2 What is the dynamic type of bob?
 - Who knows! (interface)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - Yes, since name is abstract in Person (the static type)
- Q4 Is there time travel in the call bob.name()?
 - Can be. Alice implements Person can be written after usePerson is compiled

Exercises

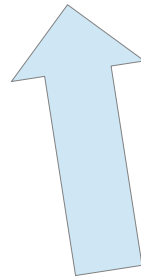
```
interface Person{
    String name();
    Cat cat();
}
record APerson(String name, Cat cat)
    implements Person {}
...
void usePerson(Person bob){
    bob.name();
}
```

- That is: the existence of 'APerson' changes none of the answers.
- Open world mindset: we do not know how many classes extends 'Person'!

- Q1 What is the static type of bob?
 - Person
- Q2 What is the dynamic type of bob?
 - Who knows! (interface)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - Yes, since name is abstract in Person (the static type)
- Q4 Is there time travel in the call bob.name()?
 - Can be. Alice implements Person can be written after usePerson is compiled

Exercises

```
interface Person{
    String name();
    Cat cat();
}
record APerson(String name, Cat cat)
    implements Person {}
...
void usePerson(APerson bob){
    bob.name();
}
```



- Q1 What is the static type of bob?
 - ????
- Q2 What is the dynamic type of bob?
 - ????
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - ????
 - ????
- Q4 Is there time travel in the call bob.name()?
 - ????

Exercises

```
interface Person{
    String name();
    Cat cat();
}
record APerson(String name, Cat cat)
    implements Person {}
...
void usePerson(APerson bob){
    bob.name();
}
```

- Q1 What is the static type of bob?
 - APerson
- Q2 What is the dynamic type of bob?
 - ????
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - ????
 - ????
- Q4 Is there time travel in the call bob.name()?
 - ????

Exercises

```
interface Person{
    String name();
    Cat cat();
}
record APerson(String name, Cat cat)
    implements Person {}
...
void usePerson(APerson bob){
    bob.name();
}
```

- Q1 What is the static type of bob?
 - APerson
- Q2 What is the dynamic type of bob?
 - APerson (records are final)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - ????
 - ????
- Q4 Is there time travel in the call bob.name()?
 - ????

Exercises

```
interface Person{
    String name();
    Cat cat();
}
record APerson(String name, Cat cat)
    implements Person {}
...
void usePerson(APerson bob){
    bob.name();
}
```

- Q1 What is the static type of bob?
 - APerson
- Q2 What is the dynamic type of bob?
 - APerson (records are final)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - Yes*, the implementation could be either in Person or APerson
- Q4 Is there time travel in the call bob.name()?
 - ????

Exercises

```
interface Person{
    String name();
    Cat cat();
}
record APerson(String name, Cat cat)
    implements Person {}
...
void usePerson(APerson bob){
    bob.name();
}
```

- Q1 What is the static type of bob?
 - APerson
- Q2 What is the dynamic type of bob?
 - APerson (records are final)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - Yes*, the implementation could be either in Person or APerson
- Q4 Is there time travel in the call bob.name()?
 - No, name() is final / APerson is final

Exercises

```
interface Person{
    String name();
    Cat cat();
}
record APerson(String name, Cat cat)
    implements Person {}
...
void usePerson(APerson bob){
    ((Person)bob).name();
}
```

- Q1 What is the static type of bob?
 - APerson
- Q2 What is the dynamic type of bob?
 - APerson (records are final)
- Q3 Is there dynamic dispatch in the call `((Person)bob).name()` ?
 - Yes*, the implementation could be either in Person or APerson
- Q4 Is there time travel in the call `bob.name()`?
 - No, name() is final / APerson is final

Exercises

```
interface Person{  
    default String name(){return "Adam";}  
    Cat cat();  
}  
record APerson(String name, Cat cat)  
    implements Person {}  
  
...  
void usePerson(APerson bob){  
    bob.name();  
}
```

- That is: Person.name having a default implementation changes nothing since it is overridden in the static type

*since name() is overridden in the final APerson, the dynamic dispatch may be optimized away

- Q1 What is the static type of bob?
 - APerson
- Q2 What is the dynamic type of bob?
 - APerson (records are final)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - Yes*, the implementation could be either in Person or APerson
- Q4 Is there time travel in the call bob.name()?
 - No, name() is final / APerson is final

Exercises

```
interface Person{  
    default String name(){return "Adam";}  
    Cat cat();  
}  
record APerson(Cat cat)//Note: no name  
    implements Person {}  
  
...  
void usePerson(APerson bob){  
    bob.name();  
}
```

- Q1 What is the static type of bob?
 - APerson
- Q2 What is the dynamic type of bob?
 - APerson (records are final)
- Q3 Is there dynamic dispatch in the call bob.name() ?
 - Yes, the implementation could be either in Person or APerson
- Q4 Is there time travel in the call bob.name()?
 - No, (final) APerson must exist for usePerson to compile.

Exercises

```
interface Destiny{  
    default void accept(){  
        System.out.println("Oh, well..");  
    }  
    default int fight(){  
         this.accept();  
        return Integer.MAX_VALUE;  
    }  
}
```

- Q1 What is the static type of 'this'?
 - ????
- Q2 What is the dynamic type of 'this'?
 - ????
- Q3 Is there dynamic dispatch in the call `this.accept()` ?
 - ????
 - ????
- Q4 Is there time travel in the call `this.accept()`?
 - ????
 - ????

Exercises

```
interface Destiny{  
    default void accept(){  
        System.out.println("Oh, well..");  
    }  
    default int fight(){  
         this.accept();  
        return Integer.MAX_VALUE;  
    }  
}
```

- Q1 What is the static type of 'this'?
 - Destiny
- Q2 What is the dynamic type of 'this'?
 - ????
- Q3 Is there dynamic dispatch in the call `this.accept()` ?
 - ????
 - ????
- Q4 Is there time travel in the call `this.accept()`?
 - ????
 - ????

Exercises

```
interface Destiny{  
    default void accept(){  
        System.out.println("Oh, well..");  
    }  
    default int fight(){  
         this.accept();  
        return Integer.MAX_VALUE;  
    }  
}
```

- Q1 What is the static type of 'this'?
 - Destiny
- Q2 What is the dynamic type of 'this'?
 - Who knows
- Q3 Is there dynamic dispatch in the call `this.accept()` ?
 - ????
 - ????
- Q4 Is there time travel in the call `this.accept()`?
 - ????
 - ????

Exercises

```
interface Destiny{  
    default void accept(){  
        System.out.println("Oh, well..");  
    }  
    default int fight(){  
         this.accept();  
        return Integer.MAX_VALUE;  
    }  
}
```

- Q1 What is the static type of 'this'?
 - Destiny
- Q2 What is the dynamic type of 'this'?
 - Who knows
- Q3 Is there dynamic dispatch in the call `this.accept()` ?
 - Yes, the implementation could be in the subclass of Destiny
- Q4 Is there time travel in the call `this.accept()`?
 - ????
 - ????

Exercises

```
interface Destiny{  
    default void accept(){  
        System.out.println("Oh, well..");  
    }  
    default int fight(){  
        this.accept();  
        return Integer.MAX_VALUE;  
    }  
}
```

- Q1 What is the static type of 'this'?
 - Destiny
- Q2 What is the dynamic type of 'this'?
 - Who knows
- Q3 Is there dynamic dispatch in the call `this.accept()` ?
 - Yes, the implementation could be in the subclass of Destiny
- Q4 Is there time travel in the call `this.accept()`?
 - Who knows; it depends on the dynamic type of 'this'

The iconic set up

```
interface Paintable{ void paint(Graphics2D g); }
```

```
...  
void paintAll(List<Paintable> ps, Graphics2D g){  
    for(Paintable p:ps) { p.paint(g); }  
}
```

- This is the iconic example for Time travelling / inversion of control.
- Any kind of object can implement Paintable
- Code taking a list of **Paintable** can treat them uniformly.
- `p.paint(g)`: we purposely call a function with no exact idea of what is doing. It looks silly at first, but it is the best feature of OO.
- Dynamic dispatch works like an 'if' on an unbounded amount of options.
- If Paintable is implemented again after we complete paintAll, time travel may happen

The mightiest fish?



The mightiest fish?

```
class Fish{/*primitive vertebrate features*/}
class BonyFish extends Fish{/*skeleton made of bone*/}
class CartilaginousFish extends Fish{/*skeleton made of cartilage*/}

//CartilaginousFish includes
class Shark extends CartilaginousFish{/*Mighty*/}

//BonyFish includes
class RayFinnedFish extends BonyFish{/*most modern fish, thin fins supported by rays*/}
class Tuna extends RayFinnedFish{/*fast, powerful swimmer*/}
class Goldfish extends RayFinnedFish{/*tiny, cute, but still bony!*/}

class LobeFinnedFish extends BonyFish{/*fleshy fins with bones inside*/}
class EarlyTetrapod extends LobeFinnedFish{/*first real limbs*/}
class Amphibian extends EarlyTetrapod{/*lays eggs in water, can also breath air*/}
class Reptiliomorph extends Amphibian{/*more land-adapted, tougher skin, stronger legs*/}
class Reptile extends Reptiliomorph{/* scaly skin, lays eggs on land*/}
class Mammal extends Reptile{/*warm-blooded, produces milk*/}
class Cetacean extends Mammal{/*fully aquatic mammal*/}
class Whale extends Cetacean{/*Mighty*/}
class Human extends Mammal{/*tool-using, occasionally intelligent*/}
```

What is a fish? Fish or RayFinnedFish?

```
class Fish{/*primitive vertebrate features*/}
  class BonyFish extends Fish{/*skeleton made of bone*/}
  class CartilaginousFish extends Fish{/*skeleton made of cartilage*/}

//CartilaginousFish includes
class Shark extends CartilaginousFish{/*Mighty*/}

//BonyFish includes
class RayFinnedFish extends BonyFish{/*most modern fish, thin fins supported by rays*/}
  class Tuna extends RayFinnedFish{/*fast, powerful swimmer*/}
  class Goldfish extends RayFinnedFish{/*tiny, cute, but still bony!*/}

class LobeFinnedFish extends BonyFish{/*fleshy fins with bones inside*/}
  class EarlyTetrapod extends LobeFinnedFish{/*first real limbs*/}
  class Amphibian extends EarlyTetrapod{/*lays eggs in water, can also breath air*/}
  class Reptiliomorph extends Amphibian{/*more land-adapted, tougher skin, stronger legs*/}
  class Reptile extends Reptiliomorph{/* scaly skin, lays eggs on land*/}
  class Mammal extends Reptile{/*warm-blooded, produces milk*/}
  class Cetacean extends Mammal{/*fully aquatic mammal*/}
  class Whale extends Cetacean{/*Mighty*/}
  class Human extends Mammal{/*tool-using, occasionally intelligent*/}
```

What is a fish? Fish or RayFinnedFish?

```
Fish f= new Whale();//All good! winner on Fish  
Fish f= new Shark();//All good, but not as mighty  
RayFinnedFish f= new Tuna();// Arguably, the mightiest of this category
```

Yes, phylogenetically whales (and us) are fish.
In programming, this is called transitivity of subtyping.

Breathing water:

We lost the ability to breathe water because our ancestors crawled onto land and never looked back... until whales regretted the decision, but by then, it was too late, LSP was broken.
(LSP = Liskov Substitution Principle)

Liskov Substitution Principle

Breathing water:

We lost the ability to breathe water because our ancestors crawled onto land and never looked back... until whales regretted the decision, but by then, it was too late, LSP was broken.

Liskov Substitution Principle (LSP) states:

"A subtype should be able to replace its parent type without altering the correctness of the program."

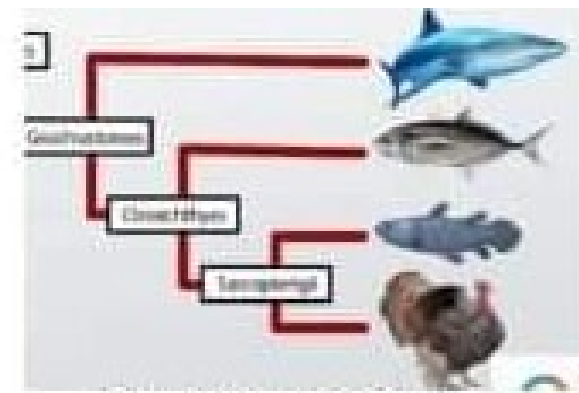
Reframed:

"If a type inherits from another, it should never lose the abilities of its parent."

In biological evolution, we do lose skills over time (like breathing water).

In good OOP design, we shouldn't lose core functionality when subclassing.

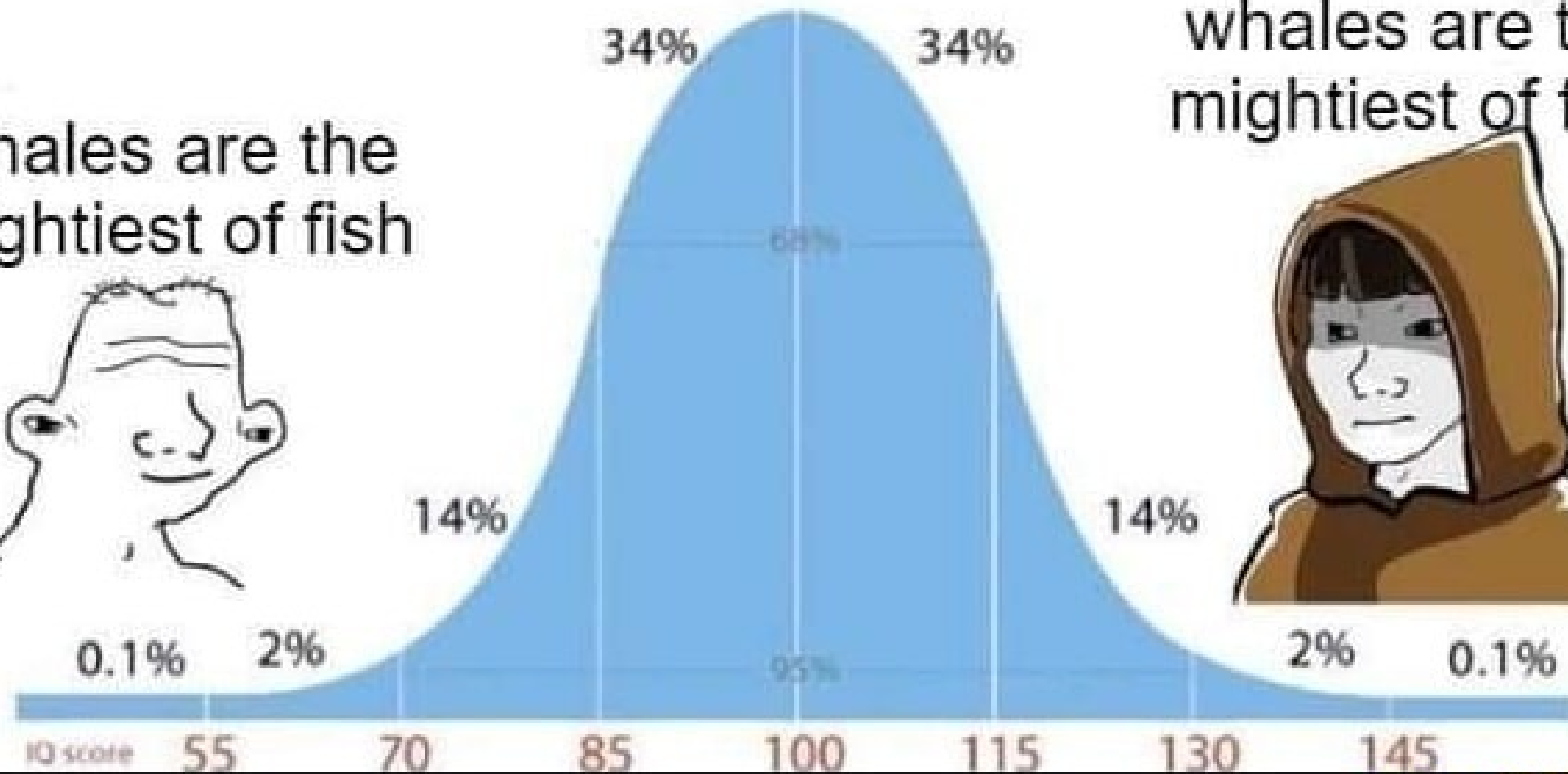
whales are
mammals, SHARKS are
the mightiest fish



Phylogenetically, if
you include sharks as "fish"
you have to include
tetrapods too since they're closer
to sarcopterygians, so really

**whales are the
mightiest of fish**

whales are the
mightiest of fish



Next! Old Java and New Java goes to battle!

